BÁO CÁO THỰC HÀNH LAB F6

Dialog, SnackBar, Menu & khung điều hướng Flutter

Scaffold · NavigationDrawer · NavigationBar · showDialog · SnackBarAction · PopupMenuButton
Bài nâng cao: NC1 IndexedStack · NC2 Dismissible · NC3 chế độ tối

Họ và tên: Nguyễn Việt Quang
MSSV: 231A290134
Lớp: CNTT – LTĐD

Flutter 3.47.3 (stable) · Dart 3.13.3 · Material 3
Máy ảo: Pixel 6 API 34 — Android 14 — 1080 × 2400 — điều hướng 3 nút
Project: F6_231A290134 (package vn.edu.vhu.ltdd.f6_231a290134)
Link GitHub: https://github.com/nvquangggit-sys/F6_231A290134
Repo được để ở chế độ Public (theo lựa chọn của sinh viên) thay cho Private + mời giảng viên.

Mục lục

Checkpoint — sản phẩm bắt buộc
1. Scaffold — bộ khung của một màn hình
2. Cấu trúc project và dữ liệu
3. Drawer và NavigationBar dùng chung một biến trạng thái
4. Hộp thoại xác nhận — showDialog trả về Future
5. Hộp thoại nhập liệu có kiểm tra hợp lệ
6. SnackBar và chức năng Hoàn tác
7. PopupMenuButton, BottomSheet và hai kiểu menu
8. Bảng 8 kịch bản kiểm thử
9. Ba câu hỏi phân tích
10. Đối chiếu với Lab A11 (Android)
11. Bài nâng cao NC1 – NC3
12. flutter analyze và flutter test
13. Thang điểm và tự đánh giá
14. Xử lý sự cố
15. Câu hỏi ôn tập
16. Kết luận và sản phẩm nộp

Checkpoint — sản phẩm bắt buộc

Checkpoint của đề

Chạy app: vuốt từ mép trái ra Drawer; chọn Thống kê thì Drawer tự đóng và thanh dưới cũng sáng mục đó; bấm + thêm việc; giữ lâu một dòng thì có hộp thoại hỏi xóa; xóa xong bấm HOÀN TÁC thì việc trở về đúng vị trí cũ.

Bài làm dựng lại toàn bộ khung ứng dụng bằng Flutter theo đúng mã nguồn mẫu của đề, sau đó bổ sung ba bài nâng cao và một số xử lý cần thiết cho dữ liệu thật (khóa ổn định cho Dismissible, tách hộp thoại nhập liệu thành StatefulWidget). App chạy trên emulator Pixel 6 API 34, ảnh trong báo cáo được chụp trực tiếp từ bản build đang chạy, không qua chỉnh sửa.

Sản phẩm	Trạng thái trong bài
Project Flutter	F6_231A290134 — GitHub public, chỉ chứa mã nguồn cần thiết; có README và lịch sử commit.
Bảng 8 kịch bản	Mục 8 — đủ 8 dòng, cột "Quan sát" đã điền bằng kết quả chạy thật.
Bảng đối chiếu A11	Mục 10 — đủ 8 dòng, kèm nhận xét Flutter gọn hơn chỗ nào, Android kiểm soát tốt chỗ nào.
3 câu hỏi phân tích	Mục 9 — trả lời kèm nguyên văn kết quả flutter analyze của từng thí nghiệm.
Video demo	docs/F6_231A290134_demo.mp4 — dài 1 phút 08 giây, đủ trình tự đề yêu cầu.
flutter analyze	Mục 12 — "No issues found!", ảnh chụp cửa sổ PowerShell thật.
Kiểm thử tự động	Mục 12 — 7 widget test trong test/widget_test.dart, flutter test: All tests passed!
Bài nâng cao	NC1 IndexedStack, NC2 Dismissible, NC3 chế độ tối (3 bài, đề yêu cầu ≥ 2).

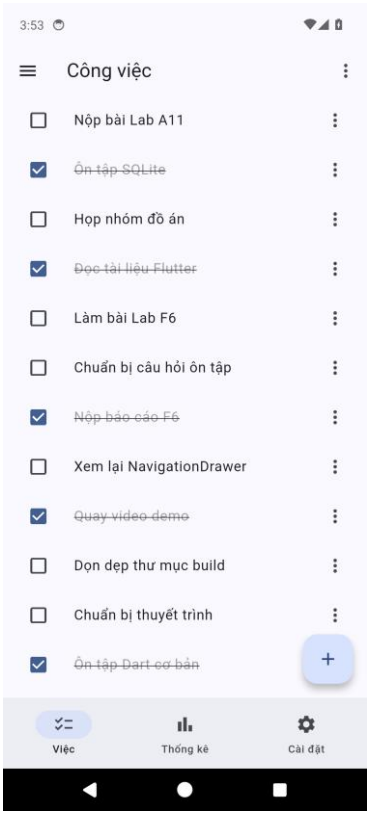
Một điểm khác với đề: đề gợi ý đặt tên project là f6_congviec, bài đặt F6_231A290134 để khớp quy ước F6_<MSSV> ở mục nộp bài. Tên hiển thị trong MaterialApp và nhãn ứng dụng Android (android:label) đều là F6_231A290134.

1. Scaffold — bộ khung của một màn hình

Toàn bộ ứng dụng nằm trong một widget Scaffold duy nhất. Năm thành phần của đề tương ứng với năm thuộc tính, không cần file XML nào:

Thuộc tính	Hiển thị ở đâu	Trong bài
appBar	Thanh trên cùng, có title và actions	AppBar(title: tên tab, actions: [PopupMenuButton ⋮]); có drawer nên AppBar tự thêm nút ba gạch.
drawer	Ngăn kéo trượt từ mép trái	NavigationDrawer: header UserAccountsDrawerHeader + 3 NavigationDrawerDestination.
body	Nội dung chính	IndexedStack(index: _tab) — NC1; mỗi tab là một hàm dựng widget.
bottomNavigationBar	Thanh dưới cùng	NavigationBar với 3 NavigationDestination: Việc / Thống kê / Cài đặt.
floatingActionButton	Nút tròn nổi góc dưới phải	FloatingActionButton dấu +, chỉ hiện ở tab Công việc (_tab == 0).

Khác biệt lớn nhất so với Android là mọi thứ nằm trong một đối tượng cấu hình thay vì một cây XML cộng nhiều lớp Java. Có drawer thì AppBar tự thêm nút ba gạch, không phải viết dòng nào; đổi tab chỉ là gán lại một biến int. Đó cũng là lý do phần đối chiếu ở Mục 10 kết luận Flutter gọn hơn hẳn ở khung màn hình.



Hình 1. Màn hình Công việc: AppBar có nút ba gạch và menu ⋮, danh sách 17 việc mẫu, NavigationBar 3 mục, FAB dấu +.

Danh sách mẫu gồm 17 việc, 6 việc đã xong (dòng gạch ngang, chữ xám). Số lượng này được chọn có chủ ý: đủ dài để danh sách cuộn được — điều kiện để thấy rõ tác dụng giữ vị trí cuộn của NC1. Khi thêm việc trong lúc kiểm thử, danh sách thành 18 việc.

2. Cấu trúc project và dữ liệu

Project tạo bằng flutter create với --org vn.edu.vhu.ltdd, sau đó thay toàn bộ lib/main.dart. Mã nguồn khoảng 470 dòng, chia thành bốn khối rõ ràng:

Khối	Nội dung
MyApp (_MyAppState)	MaterialApp, theme/darkTheme, themeMode. Giữ biến _toi cho NC3 nên MyApp là StatefulWidget; HomeShell nhận cờ này qua tham số.
CongViec	Model bất biến: id (khóa ổn định cho Dismissible), tieuDe, xong; copyWith giữ nguyên id.
HomeShell (_HomeShellState)	Toàn bộ trạng thái màn hình: _tab, _hienDaXong, _ds; các thao tác thêm / xóa / đổi trạng thái; dựng AppBar, Drawer, body, NavigationBar, FAB.
_ThemCongViecDialog	StatefulWidget riêng cho hộp thoại nhập liệu, tự quản lý và dispose TextEditingController.

Hai điểm khác mã nguồn mẫu của đề, đều xuất phát từ lỗi gặp khi chạy thật:

- Model có thêm id tăng dần. Dismissible (NC2) bắt buộc dùng key duy nhất và ổn định; nếu dùng ValueKey(tieuDe) thì hai việc trùng tên sẽ làm Flutter báo lỗi “Duplicate keys found”.
- Hộp thoại nhập liệu được tách thành StatefulWidget. Cách dispose controller ngay sau await showDialog trong mã mẫu gây red screen trên Flutter 3.47 — chi tiết ở Mục 5.3 và Mục 14.

Biến _hienDaXong quyết định danh sách hiển thị, còn _ds là dữ liệu gốc. Đây là cặp biến quan trọng nhất của bài và là chìa khóa cho câu hỏi phân tích số 1:

```
bool _hienDaXong = true;
final List<CongViec> _ds = [ /* 17 việc mẫu */ ];

/// Danh sách đang hiển thị (sau khi lọc), tách khỏi _ds là dữ liệu gốc.
List<CongViec> get _hienThi =>
    _hienDaXong ? _ds : _ds.where((c) => !c.xong).toList();
```

_hienThi không phải một biến được đồng bộ tay — nó là getter, luôn tính lại từ _ds và _hienDaXong. Nhờ vậy không thể có chuyện hai danh sách lệch nhau sau khi thêm, xóa hay đổi trạng thái.

3. Drawer và NavigationBar dùng chung một biến trạng thái

Cả hai thanh điều hướng đọc và ghi cùng biến _tab. Chọn ở Drawer thì thanh dưới sáng theo và ngược lại — một nguồn sự thật duy nhất:

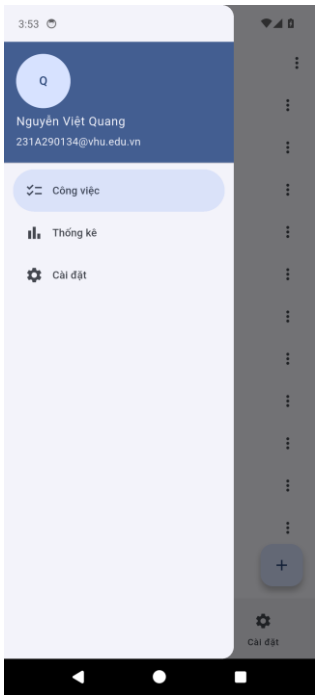
```
int _tab = 0;

drawer: NavigationDrawer(
  selectedIndex: _tab,
  onDestinationSelected: (i) {
    setState(() => _tab = i);
    Navigator.pop(context);   // đóng ngăn kéo sau khi chọn
  },
  children: const [
    UserAccountsDrawerHeader(
      accountName: Text('Nguyễn Việt Quang'),
      accountEmail: Text('231A290134@vhu.edu.vn'),
      currentAccountPicture: CircleAvatar(child: Text('Q')),
    ),
    NavigationDrawerDestination(icon: Icon(Icons.checklist), label: Text('Công việc')),
    NavigationDrawerDestination(icon: Icon(Icons.bar_chart), label: Text('Thống kê')),
    NavigationDrawerDestination(icon: Icon(Icons.settings), label: Text('Cài đặt')),
  ],
),

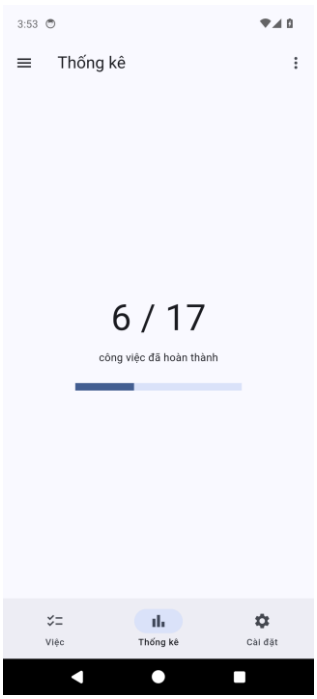
bottomNavigationBar: NavigationBar(
  selectedIndex: _tab,
  onDestinationSelected: (i) => setState(() => _tab = i),
  destinations: const [ /* Việc · Thống kê · Cài đặt */ ],
),
```

Ba chi tiết dễ sai:

- Phải gọi Navigator.pop(context) trong onDestinationSelected của Drawer, nếu không ngăn kéo cứ mở mãi và che mất tab vừa chọn.
- selectedIndex của NavigationDrawer chỉ đếm các NavigationDrawerDestination, không tính header. Header đứng đầu nhưng không làm lệch chỉ số.
- Nếu tách hai biến trạng thái riêng cho Drawer và NavigationBar thì chọn tab ở Drawer sẽ làm thanh dưới sáng sai mục — đúng thí nghiệm nhỏ trong đề. Bài chỉ dùng một biến _tab nên không có chuyện đó.



(a) Drawer mở từ mép trái, header + 3 mục



(b) Chọn Thống kê: Drawer đóng, thanh dưới sáng đúng

Hình 2. Drawer và NavigationBar cùng phản ánh biến _tab. Ảnh (b) cũng là minh chứng kịch bản 2: 6 / 17 việc đã hoàn thành.

Ảnh (b) cho thấy Drawer đã đóng ngay sau khi chọn, AppBar đổi thành “Thống kê”, mục Thống kê trên thanh dưới được tô sáng. Đây là hành vi đúng của checkpoint.

4. Hộp thoại xác nhận — showDialog trả về Future

showDialog đẩy một route mới lên ngăn xếp, nên phải await mới biết người dùng chọn gì. Kết quả là bool?: true khi bấm Xóa, false khi bấm Hủy, null khi chạm ra ngoài hoặc bấm Back.

```
/// AlertDialog xác nhận. Trả về true nếu người dùng đồng ý.
Future<bool> _xacNhanXoa(CongViec cv) async {
  final dongY = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Xác nhận xóa'),
      content: Text('Bạn có chắc muốn xóa "${cv.tieuDe}"?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Hủy'),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Xóa'),
        ),
      ],
    ),
  );
  // showDialog trả về null khi người dùng chạm ra ngoài để đóng
  return dongY ?? false;
}
```

Hàm trả về Future<bool> chứ không phải bool? nhờ dòng cuối. Nhờ vậy phía gọi không phải xử lý null lần nữa:

```
onLongPress: () async {
  final dongY = await _xacNhanXoa(cv);
  if (dongY && mounted) _xoa(cv);
},
```

Bài giữ cả hai lớp bảo vệ: hỏi trước khi xóa và cho Hoàn tác sau khi xóa. Đề yêu cầu rõ hộp thoại xác nhận; còn Hoàn tác là lưới an toàn cho cú bấm nhầm thứ hai. Cùng một hàm _xacNhanXoa được dùng lại cho Dismissible ở NC2, nên lời hỏi và cách xử lý null chỉ tồn tại ở một chỗ.

Lời hỏi nêu đúng tên việc — người dùng phải biết mình đang xóa cái gì trước khi bấm nút đỏ. Nếu chỉ ghi “Bạn có chắc không?” thì hộp thoại vô nghĩa với danh sách dài.

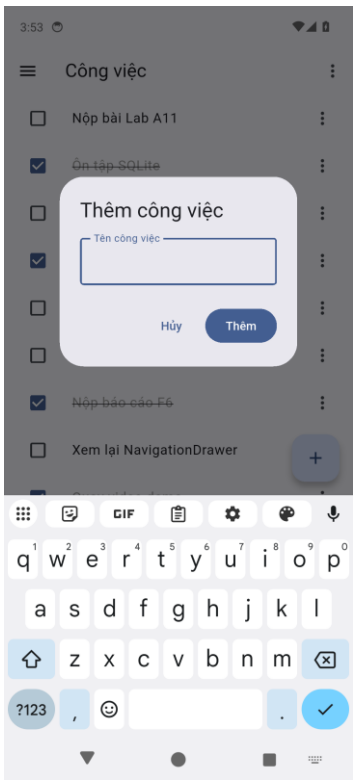
5. Hộp thoại nhập liệu có kiểm tra hợp lệ

5.1. Chỉ đóng khi dữ liệu đúng

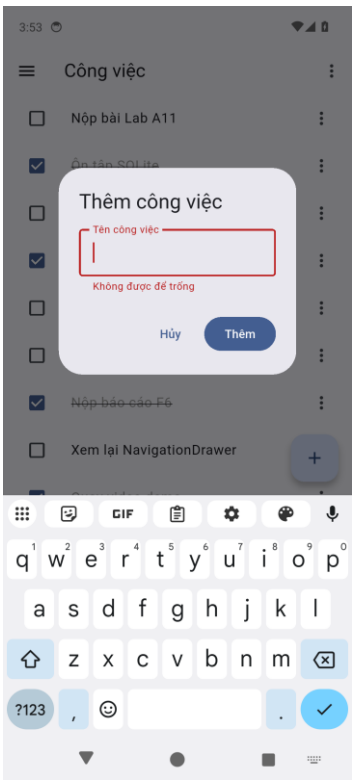
Nút trong actions không tự đóng hộp thoại; nó chỉ đóng khi ta gọi Navigator.pop. Vì vậy chỉ cần đặt Navigator.pop bên trong nhánh validate() là hộp thoại sẽ không đóng khi dữ liệu sai:

```
FilledButton(
  onPressed: () {
    // Chỉ đóng hộp thoại khi dữ liệu hợp lệ
    if (_formKey.currentState!.validate()) {
      Navigator.pop(context, _controller.text.trim());
    }
  },
  child: const Text('Thêm'),
),
```

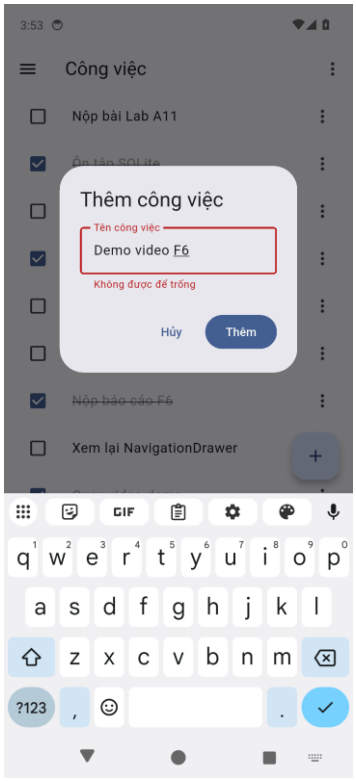
TextFormField dùng validator trả về chuỗi lỗi khi rỗng, Form tự hiện lỗi ngay dưới ô và viền ô chuyển đỏ. Dữ liệu người dùng đã gõ còn nguyên vì hộp thoại chưa bị gỡ. Đây là chỗ Flutter rõ ràng hơn Android: bên Android phải gán lại listener sau show() mới chặn được việc AlertDialog tự đóng, còn Flutter không cần mẹo gì.



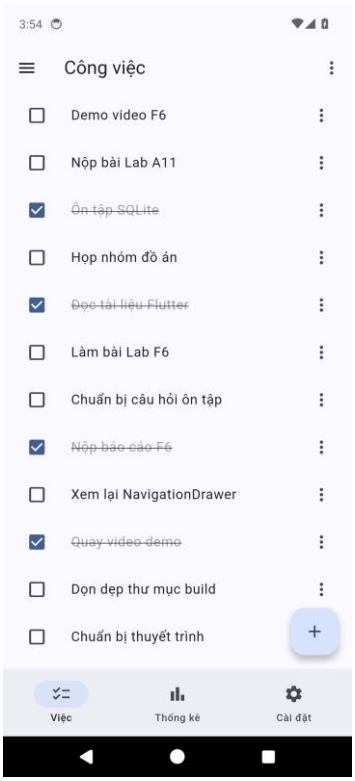
(a) Hộp thoại Thêm, ô nhập được focus



(b) Bấm Thêm khi để trống: lỗi tại ô, hộp thoại vẫn mở



(c) Đã nhập "Demo video F6"



(d) Việc mới nằm ở đầu danh sách

Hình 3. Kịch bản 3 và 4 — validate không đóng hộp thoại, sau đó thêm thành công.

5.2. Nhận kết quả và thêm vào danh sách

```
Future<void> _themCongViec() async {
  final tieuDe = await showDialog<String>(
    context: context,
    builder: (_) => const _ThemCongViecDialog(),
  );

  if (tieuDe == null || !mounted) return;
  setState(() => _ds.insert(0, CongViec(tieuDe)));
}
```

Hộp thoại trả về String? — null khi Hủy hoặc chạm ra ngoài. Kiểm tra null trước, rồi mới kiểm tra mounted, rồi mới setState. Thêm vào đầu danh sách bằng insert(0, …) đúng yêu cầu "việc mới nằm ở đầu".

5.3. Sự cố thật: dispose controller ngay sau await

Mã nguồn mẫu của đề tạo TextEditingController trong hàm, await showDialog rồi gọi controller.dispose() ngay. Trên Flutter 3.47 cách này làm app red screen ngay sau khi hộp thoại đóng:

```
Failed assertion: line 6281 pos 12: '_dependents.isEmpty': is not true.
The relevant error-causing widget was:
  Form  file:///E:/LTDiDong/F6_231A290134/lib/main.dart:...
When the exception was thrown, this was the stack:
  #2  InheritedElement.debugDeactivated (package:flutter/src/widgets/framework.dart:6281:12)
```

Nguyên nhân: tại thời điểm await trả về, route của hộp thoại mới bắt đầu đóng; TextFormField bên trong vẫn còn đăng ký phụ thuộc vào InheritedElement (Form/MediaQuery). Dispose controller ngay lúc đó làm widget đang deactivate cố gỡ đăng ký trên một đối tượng đã bị hủy. Cách sửa là để chính hộp thoại quản lý vòng đời của nó:

```
class _ThemCongViecDialog extends StatefulWidget {
  const _ThemCongViecDialog();
  @override
  State<_ThemCongViecDialog> createState() => _ThemCongViecDialogState();
}

class _ThemCongViecDialogState extends State<_ThemCongViecDialog> {
  final TextEditingController _controller = TextEditingController();
  final GlobalKey<FormState> _formKey = GlobalKey<FormState>();

  @override
  void dispose() {
    _controller.dispose();   // chạy khi widget thực sự bị gỡ khỏi cây
    super.dispose();
  }
  // … build() trả về AlertDialog như trên
}
```

Đây là ví dụ cho thấy "chạy được trên máy giảng viên" không đủ: cùng một đoạn mã có thể đúng ở phiên bản Flutter cũ và sai ở phiên bản mới. Cách sửa không phải vá bằng delay mà là đặt vòng đời cho đúng chỗ.

6. SnackBar và chức năng Hoàn tác

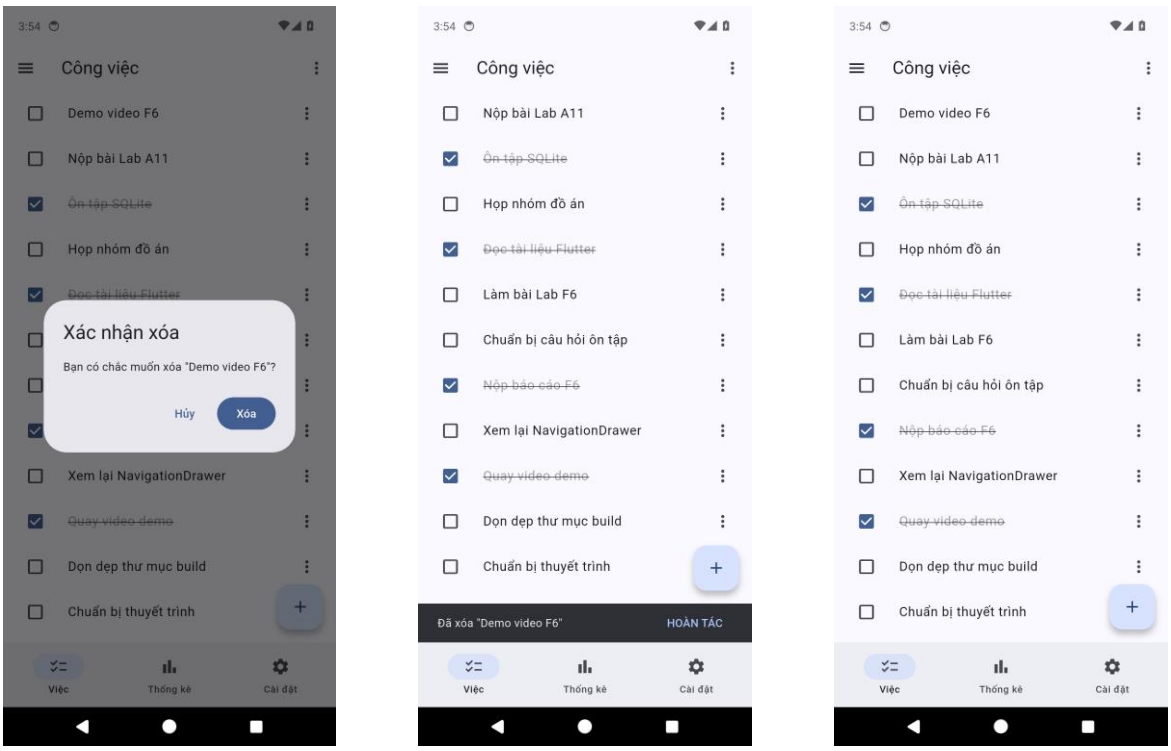
Xóa xong, đối tượng được giữ lại trong biến cục bộ và cho người dùng vài giây để cứu. Điểm mấu chốt là phải giữ cả đối tượng lẫn vị trí, nếu không việc sẽ quay về cuối danh sách:

```
void _xoa(CongViec cv) {
  final i = _ds.indexOf(cv);
  if (i < 0) return;
  setState(() => _ds.removeAt(i));

  // Gỡ SnackBar đang hiện trước khi hiện cái mới, tránh xếp hàng chồng chất
  ScaffoldMessenger.of(context)
    ..hideCurrentSnackBar()
    ..showSnackBar(
      SnackBar(
        content: Text('Đã xóa "${cv.tieuDe}"'),
        action: SnackBarAction(
          label: 'HOÀN TÁC',
          onPressed: () => setState(() => _ds.insert(i, cv)),
        ),
      ),
    );
}
```

- i được tính trên _ds (dữ liệu gốc) nên hoàn tác chèn lại đúng vị trí, kể cả khi danh sách đang ẩn việc đã xong.
- Toán tử .. (cascade) gọi hai phương thức trên cùng một đối tượng ScaffoldMessenger; hideCurrentSnackBar tránh cảnh nhiều SnackBar xếp hàng khi xóa liên tiếp.
- ScaffoldMessenger thuộc phạm vi ứng dụng, nên SnackBar không biến mất khi đổi tab. Đây là lý do dùng nó thay cho một widget SnackBar gắn cứng vào màn hình.
- Sau khi hoàn tác, nếu đang bật "Ẩn việc đã xong" mà việc vừa xóa lại là việc đã xong thì dữ liệu vẫn được phục hồi nhưng không hiện ra — đúng với bộ lọc đang bật, không phải lỗi.
- Bài xóa việc ở đầu danh sách nên ảnh sau khi hoàn tác trùng khớp ảnh trước khi xóa (Hình 3d) — đúng như kỳ vọng: dữ liệu trở về nguyên trạng. Cơ chế insert(i, cv) không phụ thuộc vị trí; xóa một việc ở giữa sẽ thấy nó quay lại đúng giữa hai việc bên cạnh.

Việc cần làm	Widget Flutter	Tương đương Android
Báo nhanh, không nút	SnackBar không có action	Toast
Báo kèm một hành động	SnackBar + SnackBarAction	Snackbar + setAction
Hỏi trước khi làm	showDialog + AlertDialog	AlertDialog.Builder
Bảng hành động trượt lên	showModalBottomSheet	BottomSheetDialog



(a) Giữ lâu: hỏi lại đúng tên việc *(b) Bấm Xóa: SnackBar + HOÀN TÁC* *(c) HOÀN TÁC: danh sách trở về đúng trạng thái trước khi xóa*

Hình 4. Kịch bản 5 và 6 — xác nhận xóa, SnackBar Hoàn tác. Ảnh (c) trùng khớp Hình 3d vì hoàn tác đưa danh sách về đúng trạng thái trước khi xóa — đó chính là điều cần chứng minh. Kịch bản 8 (chạm ra ngoài) không có ảnh riêng vì sau khi đóng hộp thoại, trạng thái trùng ảnh trước khi mở; kết quả ghi ở bảng Mục 8.

Kịch bản 8 không cần ảnh riêng cũng đủ kiểm chứng bằng mã nguồn: showDialog mặc định barrierDismissible: true, nên chạm vào vùng tối bên ngoài sẽ đóng hộp thoại và future hoàn tất với null. Nhánh if (dongY && mounted) coi null là "không đồng ý", nên việc vẫn nằm trong danh sách và không có SnackBar. Ảnh f6_huy.png trong docs/screenshots ghi lại đúng trạng thái đó — trùng với ảnh trước khi mở hộp thoại, vì không có gì thay đổi.

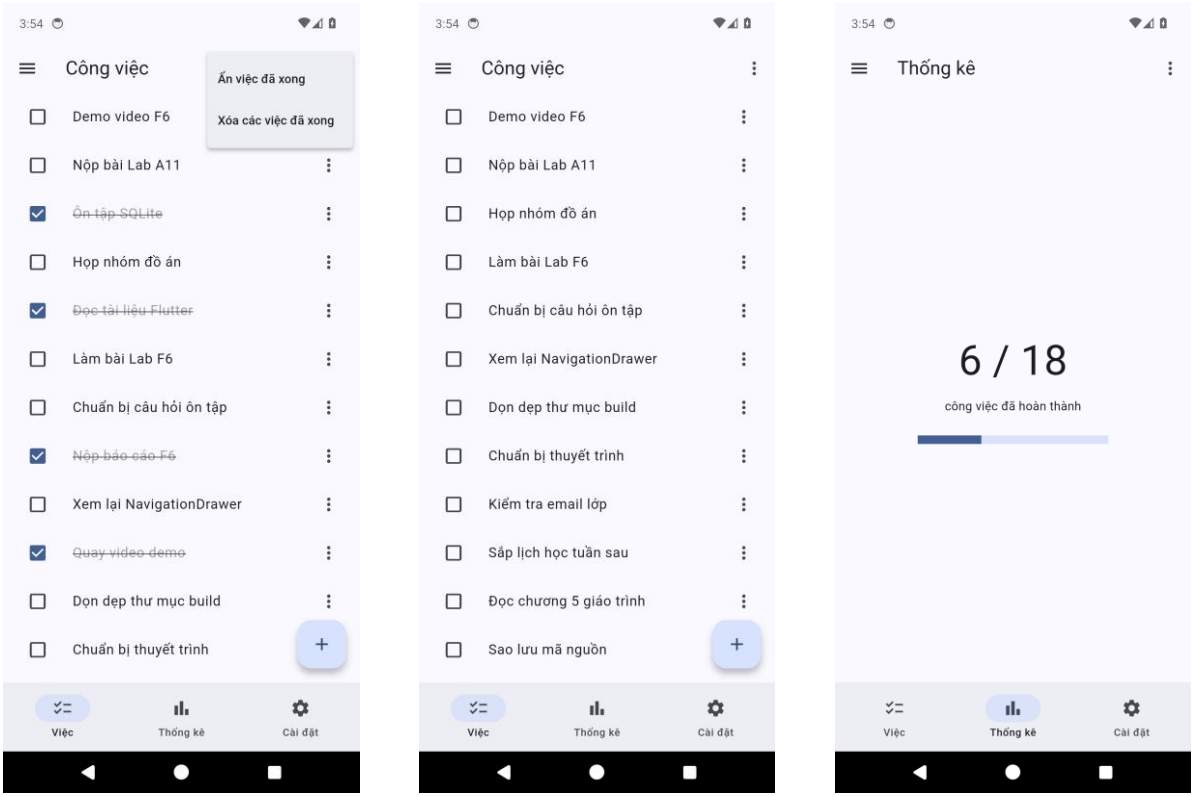
Ngược lại, nếu muốn buộc người dùng chọn rõ ràng thì đặt barrierDismissible: false; khi đó bắt buộc phải có nút Hủy, nếu không người dùng sẽ bị kẹt trong hộp thoại. Bài giữ mặc định vì thao tác xóa đã có hai lớp bảo vệ: hỏi lại trước khi xóa và SnackBar Hoàn tác sau khi xóa.

7. PopupMenuButton, BottomSheet và hai kiểu menu

Menu ba chấm là PopupMenuButton đặt trong actions của AppBar — tương đương options menu của Android. Bài dùng hai mục: đổi trạng thái lọc và xóa các việc đã xong.

```
PopupMenuButton<String>(
  onSelected: (chon) {
    if (chon == 'loc') {
      setState(() => _hienDaXong = !_hienDaXong);
    } else if (chon == 'xoa_xong') {
      setState(() => _ds.removeWhere((c) => c.xong));
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Đã xóa các việc đã xong')),
      );
    }
  },
  itemBuilder: (context) => [
    PopupMenuItem(
      value: 'loc',
      child: Text(_hienDaXong ? 'Ẩn việc đã xong' : 'Hiện việc đã xong'),
    ),
    const PopupMenuItem(
      value: 'xoa_xong',
      child: Text('Xóa các việc đã xong'),
    ),
  ],
),
```

Nhãn mục đầu đổi theo trạng thái (_hienDaXong ? 'Ẩn …' : 'Hiện …'), nên người dùng luôn thấy hành động sắp xảy ra chứ không phải trạng thái hiện tại. Đây là chi tiết nhỏ nhưng ảnh hưởng trực tiếp đến việc có bấm đúng hay không.



(a) Menu : với 2 mục, nhãn đổi theo trạng thái

(b) Sau khi ẩn: còn 12/18 việc chưa xong

(c) Thống kê vẫn đếm đủ 6/18

Hình 5. Kịch bản 7 — ẩn việc đã xong không làm thay đổi dữ liệu gốc.

7.1. Menu của một dòng: BottomSheet

Chạm vào một dòng mở bảng hành động trượt lên. Bảng dùng Column với mainAxisSize.min để không chiếm hết chiều cao, và mọi thao tác đóng bảng đều dùng sheetContext của builder:

```
void _moBangHanhDong(CongViec cv) {
  showModalBottomSheet(
    context: context,
    showDragHandle: true,
    builder: (sheetContext) => SafeArea(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          ListTile(title: Text(cv.tieuDe, style: const TextStyle(fontWeight:
FontWeight.bold))),
          const Divider(height: 1),
          ListTile(
            leading: Icon(cv.xong ? Icons.undo : Icons.check),
            title: Text(cv.xong ? 'Đánh dấu chưa xong' : 'Đánh dấu đã xong'),
            onTap: () {
              Navigator.pop(sheetContext);
              _doiTrangThai(cv);
            },
          ),
          ListTile(
            leading: const Icon(Icons.delete_outline, color: Colors.red),
            title: const Text('Xóa', style: TextStyle(color: Colors.red)),
            onTap: () async {
              Navigator.pop(sheetContext);
              final dongY = await _xacNhanXoa(cv);
              if (dongY && mounted) _xoa(cv);
            },
          ),
        ],
      ),
    ),
  );
}
```

Kiểu menu	Cách làm trong bài	Dùng khi
Menu màn hình	PopupMenuButton trong actions của AppBar	Vài hành động chung: lọc, xóa hàng loạt
Menu một dòng	showModalBottomSheet, mở bằng onTap	Nhiều hành động, cần mô tả và biểu tượng rõ
Giữ lâu	onLongPress mở thẳng hộp thoại xóa	Lối tắt cho hành động nguy hiểm nhất

- Mọi thao tác đóng bảng đều dùng sheetContext — context do builder cấp — chứ không dùng context của màn hình: chỉ context đó mới trỏ đúng lớp phủ đang mở, nên Navigator.pop không đóng nhầm màn hình phía dưới.
- Bảng được đóng trước rồi mới await _xacNhanXoa(cv). Nếu mở hộp thoại xác nhận lúc bảng còn hiện, hai lớp phủ chồng lên nhau và người dùng phải bấm hai lần mới thoát ra được.
- SafeArea giữ nội dung không bị thanh điều hướng che, showDragHandle cho tay cầm kéo quen thuộc, còn mainAxisSize.min giữ bảng cao đúng bằng nội dung thay vì chiếm trọn màn hình như một trang mới.
- Đây là kiểu menu “một dòng” phù hợp khi hành động cần biểu tượng và mô tả rõ; menu ba chấm ở AppBar chỉ nên dành cho vài hành động chung của cả danh sách.

8. Bảng 8 kịch bản kiểm thử

Thiết bị: Pixel 6 API 34, Android 14, 1080 × 2400, chế độ sáng, điều hướng 3 nút. Kịch bản 1–4 chạy lại từ trạng thái sạch (am force-stop rồi mở lại app) trước khi chụp; kịch bản 5–8 nối tiếp trong cùng một phiên (thêm "Demo video F6" ở kịch bản 4, rồi xóa và hoàn tác ở 5–6) để thấy đúng hành vi thật.

#	Thao tác	Kết quả đúng phải thấy	Quan sát
1	Mở app, vuốt từ mép trái	Drawer trượt ra, đủ ba mục, phần đầu hiện tên	Drawer mở; header "Nguyễn Việt Quang / 231A290134@vhu.edu.vn"; 3 mục Công việc, Thống kê, Cài đặt; mục Công việc đang được tô sáng — Hình 2a
2	Chọn Thống kê trong Drawer	Drawer tự đóng, sang tab Thống kê, thanh dưới sáng mục đó	Drawer đóng ngay; AppBar đổi "Thống kê"; thanh dưới sáng mục Thống kê; hiện 6 / 17 — Hình 2b
3	Bấm + rồi bấm Thêm khi để trống	Báo lỗi dưới ô nhập, hộp thoại không đóng	Ô nhập viền đỏ, dòng "Không được để trống" ngay dưới ô; hộp thoại và bàn phím vẫn còn nguyên — Hình 3b
4	Nhập tên việc rồi bấm Thêm	Việc mới nằm ở đầu danh sách	"Demo video F6" xuất hiện đầu danh sách, tổng 18 việc; hộp thoại đóng — Hình 3d
5	Giữ lâu một dòng rồi bấm Xóa	Hộp thoại hỏi lại nêu đúng tên việc	"Bạn có chắc muốn xóa "Demo video F6"?" với hai nút Hủy / Xóa — Hình 4a
6	Bấm HOÀN TÁC trên SnackBar	Việc trở lại đúng vị trí cũ	SnackBar "Đã xóa …" + HOÀN TÁC (Hình 4b); bấm HOÀN TÁC thì danh sách trở về đúng 18 việc như trước khi xóa, ảnh trùng khớp Hình 3d — Hình 4c
7	Menu ⋮ → Ẩn việc đã xong	Chỉ còn việc chưa xong; tab Thống kê vẫn đếm đủ cả hai	Danh sách còn 12/18 dòng, không còn dòng gạch ngang (Hình 5b); Thống kê vẫn 6 / 18, thanh tiến trình không đổi (Hình 5c)
8	Chạm ra ngoài hộp thoại xác nhận	Hộp thoại đóng, không xóa gì cả	Hộp thoại đóng, danh sách vẫn đủ 18 việc, không có SnackBar — không có ảnh riêng vì trạng thái trùng ảnh trước khi mở hộp thoại

Hai quan sát đáng chú ý. Thứ nhất, ở kịch bản 3 hộp thoại không đóng và bàn phím vẫn còn — điều này chỉ đúng vì Navigator.pop nằm trong nhánh validate(). Thứ hai, ở kịch bản 7 số liệu Thống kê không đổi (6/18) trong khi danh sách chỉ còn 12 dòng; đây chính là bằng chứng cho câu trả lời ở Mục 9.1.

Một chi tiết dễ gây thắc mắc khi đọc bảng: kịch bản 2 ghi 6 / 17 còn các hình sau ghi 6 / 18. Dữ liệu mẫu có 17 việc; kịch bản 4 thêm việc "Demo video F6" nên từ đó danh sách có 18 việc. Số việc đã xong vẫn là 6 vì việc mới chưa được đánh dấu, nên 6/17 và 6/18 đều đúng ở thời điểm tương ứng — không phải số liệu chép nhầm.

9. Ba câu hỏi phân tích

9.1. Vì sao tab Thống kê vẫn đếm đủ cả việc đã ẩn?

Vì tab Thống kê tính trên _ds — dữ liệu gốc — chứ không tính trên _hienThi. Biến _hienDaXong chỉ quyết định danh sách nào được dựng ra màn hình:

```
// Thống kê luôn đếm trên _ds (dữ liệu gốc), không phụ thuộc bộ lọc hiển thị
final xong = _ds.where((c) => c.xong).length;
final tong = _ds.length;

// Còn danh sách hiển thị là:
List<CongViec> get _hienThi =>
    _hienDaXong ? _ds : _ds.where((c) => !c.xong).toList();
```

Nói cách khác: _ds là biến gốc (nguồn sự thật), _hienThi là biến dẫn xuất chỉ phục vụ hiển thị. "Ẩn việc đã xong" không xóa việc nào — nó chỉ đổi cờ _hienDaXong, và getter lọc ra một danh sách mới. Dữ liệu gốc vẫn nguyên 6 việc đã xong trên 18 việc, nên Thống kê vẫn là 6/18.

Nếu ai đó viết Thống kê dựa trên _hienThi thì khi ẩn sẽ thấy 6/12 — sai về bản chất, vì người dùng chưa hoàn thành thêm việc nào, chỉ đang thay đổi cách nhìn. Ngược lại, nếu Thống kê đếm trên danh sách đã lọc thì bật/tắt bộ lọc sẽ làm tỉ lệ hoàn thành nhảy loạn, và bài toán "ẩn/hiện" trở thành bài toán sửa dữ liệu.

Đây cũng là lý do _hienThi là getter chứ không phải một List được gán lại thủ công: getter luôn tính từ _ds, nên không thể lệch. Bài đã kiểm chứng bằng ảnh: 6/17 trước khi thêm việc, 6/18 sau khi thêm và hoàn tác, và vẫn 6/18 sau khi ẩn 6 việc đã xong.

9.2. Nếu viết if (dongY) thay vì return dongY ?? false thì sao?

showDialog<bool> trả về bool?, vì người dùng có thể đóng hộp thoại mà không bấm nút nào (chạm ra ngoài hoặc bấm Back). Giá trị đó không dùng trực tiếp như bool được. Thí nghiệm trên chính project, đổi if (dongY == true && mounted) thành if (dongY && mounted) trong _tabCaiDat, trình phân tích báo nguyên văn:

```
Analyzing F6_231A290134...

  error - A nullable expression can't be used as a condition. Try checking that the value isn't
  'null' before using it as a condition - lib\main.dart:401:17 - unchecked_use_of_nullable_value

1 issue found. (ran in 2.1s)
```

Nếu đổi ngay trong _xacNhanXoa thành return dongY; thì lỗi ở tầng kiểu trả về:

```
  error - A value of type 'bool?' can't be returned from the method '_xacNhanXoa' because it has
  a return type of 'Future<bool>' - lib\main.dart:152:12 - return_of_invalid_type
```

Cả hai đều là lỗi biên dịch, không phải cảnh báo: chương trình không chạy được. Cách đúng là dongY ?? false (null nghĩa là không đồng ý) hoặc if (dongY == true). Bài chọn ?? false vì nó gom null và false về cùng một nghĩa "không xóa", và trả về Future<bool> để phía gọi không phải xử lý null lần nữa. Viết if (dongY == true) cũng đúng nhưng dễ bị sửa nhầm thành if (dongY) về sau.

9.3. Bỏ if (dongY && mounted) rồi chạy flutter analyze

Kết quả thí nghiệm phụ thuộc vào việc context được dùng ở đâu. Với code hiện tại, _xoa() tự gọi ScaffoldMessenger.of(context) bên trong thân hàm, nên bỏ kiểm tra mounted ở phía gọi cho ra:

```
Analyzing F6_231A290134...
No issues found! (ran in 2.6s)
```

Không có cảnh báo — nhưng đây là điểm quan trọng nhất của câu hỏi: analyzer không nhìn xuyên qua lời gọi hàm. Nó chỉ thấy await _xacNhanXoa(cv) rồi if (dongY) _xoa(cv); việc dùng context nằm trong _xoa, ngoài tầm phân tích luồng dữ liệu. Để thấy cảnh báo thật, dùng context trực tiếp ngay sau await:

```
onLongPress: () async {
  final dongY = await _xacNhanXoa(cv);
  if (dongY) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Da xoa ${cv.tieuDe}')),
    );
  }
},
```

Khi đó flutter analyze báo nguyên văn:

```
Analyzing F6_231A290134...

  info - Don't use 'BuildContext's across async gaps. Try rewriting the code to not use the
  'BuildContext', or guard the use with a 'mounted' check - lib\main.dart:334:38 -
  use_build_context_synchronously

1 issue found. (ran in 1.7s)
```

Giải thích: sau await, widget có thể đã bị gỡ khỏi cây (người dùng thoát màn hình, app bị đóng, route bị pop). Khi đó ScaffoldMessenger.of(context) ném lỗi "Looking up a deactivated widget's ancestor is unsafe". Cảnh báo của analyzer là lời nhắc dùng mounted để chắc chắn State còn sống trước khi chạm vào context.

Bài giữ nguyên if (dongY && mounted) dù với code hiện tại analyzer im lặng. Lý do: tính đúng đắn lúc chạy không phụ thuộc vào việc analyzer có nhìn thấy hay không. "Không có cảnh báo" chỉ có nghĩa là công cụ chưa phát hiện, không có nghĩa là đoạn mã an toàn. Đây là bài học chung khi dùng công cụ phân tích tĩnh.

> **Ghi chú về cách chạy thí nghiệm**
>
> Mỗi thí nghiệm đều sửa trực tiếp lib/main.dart, chạy flutter analyze, rồi khôi phục từ bản sao. Kết quả đầy đủ ở docs/logs/phan_tich.txt; SHA-256 của main.dart sau khi khôi phục trùng với bản trước thí nghiệm. Trạng thái cuối cùng của project: No issues found!

10. Đối chiếu với Lab A11 (Android)

Bảng dưới điền theo đúng 8 dòng của đề, cột Flutter là cách bài đã làm chứ không phải cách làm chung chung:

Việc cần làm	Android (Java + XML)	Flutter (Dart) — trong bài
Menu của màn hình	res/menu/*.xml + onCreateOptionsMenu	PopupMenuButton trong actions của AppBar; itemBuilder dựng 2 mục, onSelected xử lý bằng if–else
Menu của một dòng	registerForContextMenu + ContextMenuInfo	showModalBottomSheet mở bằng onTap; onLongPress mở thẳng hộp thoại xóa
Hộp thoại xác nhận	AlertDialog.Builder + setPositiveButton	showDialog<bool> + AlertDialog; hai nút gọi Navigator.pop(context, false/true)
Nhận kết quả hộp thoại	Callback trong setPositiveButton	await showDialog<bool>(…); xử lý null bằng dongY ?? false
Báo nhanh	Toast.makeText(...).show()	SnackBar không có action (Flutter không có Toast sẵn)
Hoàn tác	Snackbar + setAction	SnackBarAction(label: 'HOÀN TÁC') chèn lại _ds.insert(i, cv)
Thanh điều hướng dưới	BottomNavigationView + file menu	NavigationBar + 3 NavigationDestination, selectedIndex: _tab
Ngăn kéo trái	DrawerLayout + NavigationView	drawer: NavigationDrawer(selectedIndex: _tab, onDestinationSelected: …)

10.1. Flutter làm gọn hơn hẳn ở đâu?

Khung màn hình. Bên Android, một màn hình có drawer + thanh dưới + FAB cần DrawerLayout, NavigationView, BottomNavigationView, một file menu cho thanh dưới, FloatingActionButton trong XML, rồi code ánh xạ từng id và tự đồng bộ trạng thái chọn giữa drawer và thanh dưới. Bên Flutter, tất cả là thuộc tính của một Scaffold; AppBar tự thêm nút ba gạch khi có drawer; hai thanh điều hướng cùng đọc một biến int. Không có findViewById, không có adapter cho navigation, không có file menu riêng.

Việc nhận kết quả hộp thoại cũng gọn hơn: showDialog trả về Future, await là có kết quả, không cần gán listener hay giữ tham chiếu tới Dialog để đọc trạng thái sau khi đóng.

10.2. Android kiểm soát tốt hơn ở đâu?

Menu và chuỗi tách khỏi code. res/menu/*.xml khai báo cấu trúc menu, strings.xml chứa toàn bộ chữ, nên người làm giao diện và người viết logic có thể làm song song; dịch sang ngôn ngữ khác chỉ cần thêm một file values-xx/strings.xml. Android còn cho cấu hình rất chi tiết cách hiển thị: showAsAction, actionViewClass (SearchView), actionLayout, groupId, thứ tự ưu tiên. Bên Flutter, menu là code Dart; muốn đa ngôn ngữ phải thêm package và tự tách chuỗi, còn muốn thay giao diện menu phải sửa code.

Hệ quả thực tế: với app nhỏ, cách của Flutter nhanh hơn rõ rệt; với app nhiều màn hình, nhiều ngôn ngữ và nhiều người làm, cách tách resource của Android dễ bảo trì hơn. Đây là đánh đổi, không phải cái nào "tốt hơn" tuyệt đối.

11. Bài nâng cao NC1 – NC3

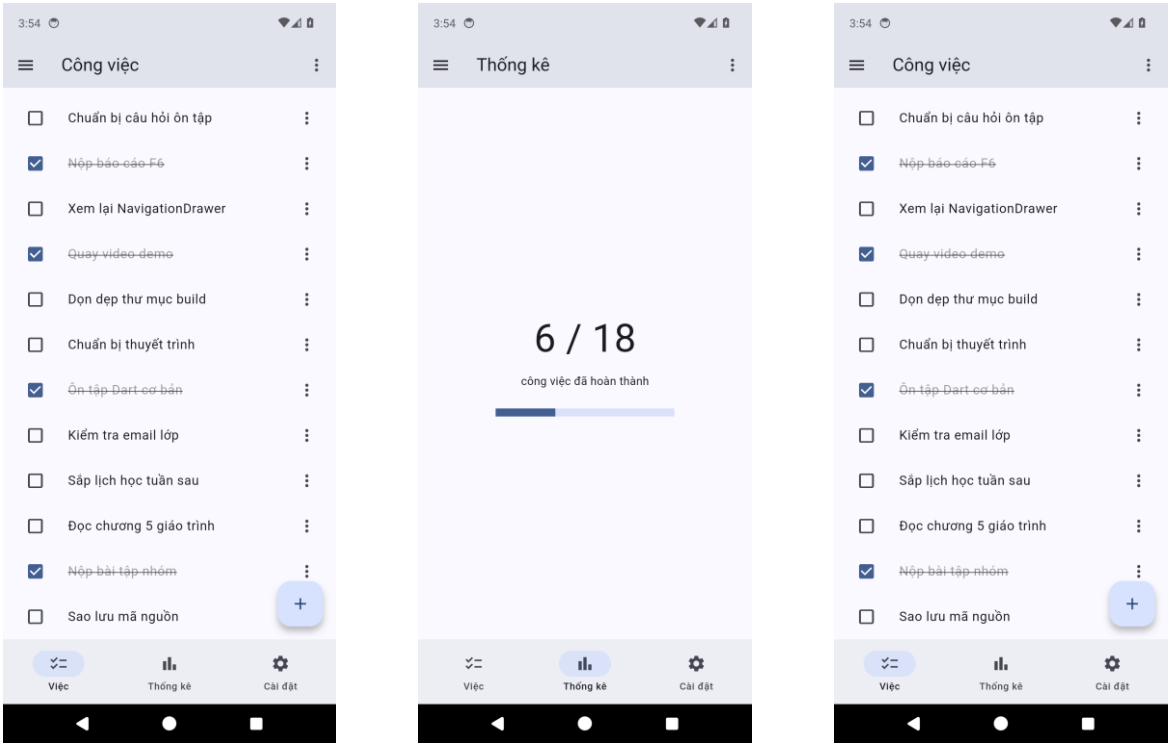
Bài chọn NC1, NC2 và NC3 — vượt yêu cầu tối thiểu 2 bài. Không chọn NC4 (showDatePicker) vì thêm hạn chót kéo theo đổi model, định dạng ngày và xử lý null/mounted ở một luồng mới; ba bài đã chọn đều trực tiếp cải thiện trải nghiệm của chính các thao tác bắt buộc.

11.1. NC1 — IndexedStack giữ nguyên vị trí cuộn

```
// NC1: IndexedStack giữ nguyên trạng thái (vị trí cuộn) của từng tab
body: IndexedStack(
  index: _tab,
  children: [
    _tabCongViec(),
    _tabThongKe(),
    _tabCaiDat(),
  ],
),
```

IndexedStack dựng sẵn cả ba tab và chỉ hiện tab có index tương ứng; hai tab còn lại vẫn nằm trong cây widget, nên ScrollController và vị trí cuộn của ListView được giữ nguyên. Với switch expression, mỗi lần đổi tab widget cũ bị hủy và tab mới được dựng từ đầu — danh sách quay về đầu. Đánh đổi là bộ nhớ: cả ba tab tồn tại cùng lúc. Với ba tab nhẹ như bài này, đổi bộ nhớ lấy trải nghiệm là hợp lý; nếu một tab có ListView ảnh lớn thì nên cân nhắc lại.

Cách kiểm chứng: cuộn danh sách xuống cuối, sang tab Thống kê rồi quay lại tab Công việc. Danh sách vẫn ở đúng vị trí cũ, không nhảy về đầu.



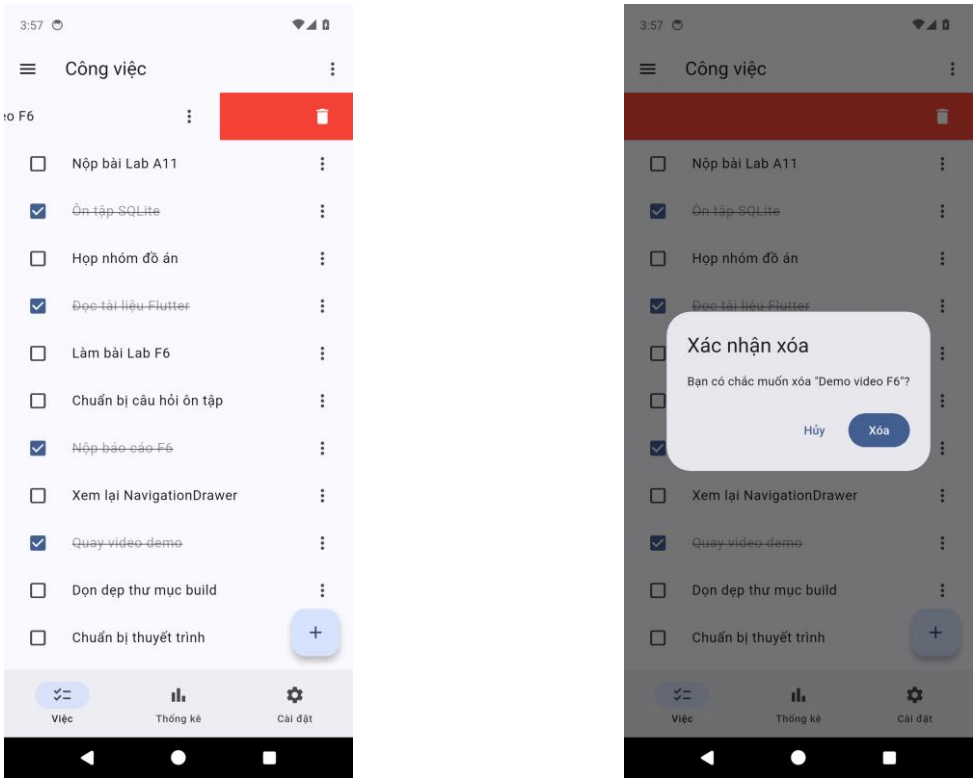
(a) Cuộn xuống cuối *(b) Sang tab Thống kê* *(c) Quay lại: vẫn đúng vị trí*

Hình 6. NC1 — IndexedStack giữ vị trí cuộn; ảnh (a) và (c) giống nhau.

11.2. NC2 — Dismissible vuốt sang trái để xóa

```
return Dismissible(
  key: ValueKey(cv.id),
  direction: DismissDirection.endToStart,
  confirmDismiss: (_) => _xacNhanXoa(cv),
  onDismissed: (_) => _xoa(cv),
  background: Container(
    color: Colors.red,
    alignment: Alignment.centerRight,
    padding: const EdgeInsets.symmetric(horizontal: 24),
    child: const Icon(Icons.delete, color: Colors.white),
  ),
  child: ListTile( /* checkbox, tiêu đề, menu … */ ),
);
```

Key phải duy nhất và ổn định, nên model có thêm id tăng dần thay vì dùng tiêu đề (hai việc trùng tên là chuyện bình thường). direction: endToStart giới hạn chỉ vuốt từ phải sang trái. confirmDismiss trả về Future<bool?> từ chính _xacNhanXoa, nên vuốt cũng phải hỏi lại; nếu người dùng bấm Hủy, Dismissible tự trượt dòng về chỗ cũ. Khi đồng ý, onDismissed gọi _xoa — vẫn là hàm cũ, nên vẫn có SnackBar và HOÀN TÁC. Nhờ vậy NC2 không tạo ra con đường xóa thứ hai thiếu bảo vệ.



(a) Đang vuốt: nền đỏ + biểu tượng thùng rác *(b) Vẫn hỏi lại trước khi xóa*

Hình 7. NC2 — Dismissible dùng chung hộp thoại xác nhận với giữ lâu.

11.3. NC3 — chế độ tối bằng Switch

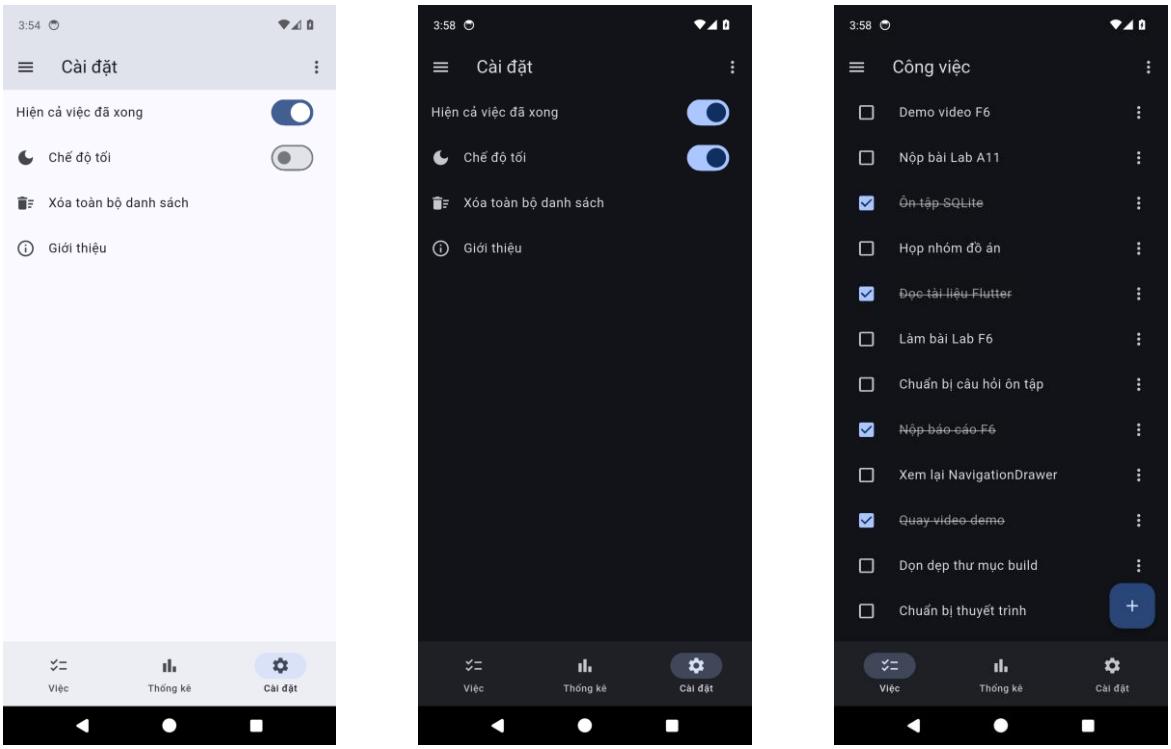
```
class _MyAppState extends State<MyApp> {
  // NC3: chế độ tối – trạng thái đặt ở widget gốc để MaterialApp đổi themeMode
  bool _toi = false;

  @override
  Widget build(BuildContext context) {
    final seed = const Color(0xFF0468D7);
    return MaterialApp(
      theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor: seed)),
      darkTheme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: seed, brightness: Brightness.dark),
      ),
      themeMode: _toi ? ThemeMode.dark : ThemeMode.light,
      home: HomeShell(toi: _toi, onDoiToi: (v) => setState(() => _toi = v)),
    );
  }
}

// Trong tab Cài đặt:
SwitchListTile(
  secondary: const Icon(Icons.dark_mode),
  title: const Text('Chế độ tối'),
  value: widget.toi,
  onChanged: widget.onDoiToi,
),
```

themeMode phải nằm trên MaterialApp, không phải trong từng màn hình: MaterialApp là nơi chọn ThemeData, nên đổi themeMode là đổi toàn bộ cây widget. Vì vậy trạng thái _toi được đưa lên MyApp (StatefulWidget), còn HomeShell nhận cờ và callback qua tham số. Cách khác là

ValueNotifier + ValueListenableBuilder, nhưng với một biến bool thì StatefulWidget ở gốc là đủ và ít lớp hơn.



(a) Tab Cài đặt: công tắc đang tắt *(b) Bật: tab Cài đặt chuyển tối* *(c) Danh sách theo theme tối*

Hình 8. NC3 — chế độ tối áp dụng cho AppBar, danh sách, thanh dưới và hộp thoại.

12. flutter analyze và flutter test

Chạy trong thư mục project trước khi nộp. Cả hai lệnh dưới đây chạy trên bản cuối cùng:

```
PS E:\LTDiDong\F6_231A290134> flutter analyze
Analyzing F6_231A290134...
No issues found!
```

Không có error, warning hay info (dòng thời gian "ran in …s" do công cụ in kèm và thay đổi mỗi lần chạy). Ảnh dưới là cửa sổ PowerShell thật chạy đúng lệnh này, không phải ảnh dàn dựng. Nội dung đầy đủ cũng được lưu ở docs/logs/flutter_analyze.txt.



Hình 9. Kết quả flutter analyze: No issues found! (ảnh cắt phần chữ của cửa sổ console).

Project còn có 7 widget test trong test/widget_test.dart. Đây là kiểm thử tự động chạy trên cùng mã nguồn, không phải ảnh chụp (kết quả đầy đủ ở docs/logs/flutter_test.txt):

```
PS E:\LTDiDong\F6_231A290134> flutter test
00:03 +7: All tests passed!
```

Bảy bài kiểm thử lần lượt kiểm tra: khung Scaffold có NavigationBar và FAB; tab Thống kê đếm 6/17 trên dữ liệu gốc; hộp thoại thêm việc báo lỗi và không đóng khi để trống; thêm việc mới vào đầu danh sách; giữ lâu để xóa rồi HOÀN TÁC đưa việc về đúng vị trí; menu ẩn việc đã xong mà

Thống kê không đổi; bật chế độ tối đổi themeMode của MaterialApp. Nhờ vậy các hành vi quan trọng nhất được kiểm tra lại sau mỗi lần sửa mã.

Trong quá trình làm, analyzer từng báo lỗi ở các thí nghiệm của Mục 9 và từng báo lỗi thật khi hộp thoại nhập liệu còn dùng cách dispose sai. Bản nộp là bản đã sửa, không phải bản có cảnh báo bị bỏ qua.

13. Thang điểm và tự đánh giá

Tiêu chí	Điểm	Minh chứng trong bài
Scaffold đủ appBar, drawer, NavigationBar, FAB	2,0	Mục 1; Hình 1; mã nguồn dòng 207–293
Drawer và thanh dưới dùng chung một biến trạng thái	1,0	Mục 3; Hình 2a–2b
Dialog xác nhận, xử lý đúng trường hợp null	1,5	Mục 4; Hình 4a; return dongY ?? false
Dialog nhập liệu có validate, không đóng khi sai	1,5	Mục 5; Hình 3a–3c
SnackBar Hoàn tác trả về đúng vị trí cũ	1,0	Mục 6; Hình 4b–4c; insert(i, cv)
PopupMenuButton lọc danh sách hoạt động đúng	1,0	Mục 7; Hình 5a–5c
Bảng 8 kịch bản + bảng đối chiếu A11	1,0	Mục 8 và Mục 10
Bài nâng cao (1 bài × 1,0)	1,0	NC1 + NC2 + NC3 (Mục 11); đề yêu cầu ≥ 2 bài
Tổng	10	

Tự đánh giá: phần bắt buộc hoàn thành đủ và đã chạy trên máy ảo; ba bài nâng cao đều có ảnh minh chứng. Điểm trừ tự thấy là video demo chưa có thuyết minh bằng giọng nói (chỉ có thao tác), và dữ liệu chưa được lưu qua lần mở app — cả hai nằm ngoài yêu cầu của đề nhưng sẽ là việc tiếp theo nếu có thời gian.

14. Xử lý sự cố

Bảy dòng đầu là bảng phụ lục của đề; bốn dòng cuối là sự cố thực tế gặp trong bài và cách đã xử lý:

Hiện tượng	Nguyên nhân	Cách xử lý
Drawer không tự đóng sau khi chọn	Quên Navigator.pop(context) trong onDestinationSelected	Thêm vào cuối callback — Mục 3
Thanh dưới và Drawer sáng khác nhau	Dùng hai biến trạng thái riêng	Dùng chung một biến _tab cho cả hai
Lỗi khi dùng kết quả của showDialog	Chưa xử lý null	dongY ?? false hoặc if (dongY == true) — Mục 4, 9.2
Cảnh báo use_build_context_synchronously	Dùng context sau await mà chưa kiểm tra mounted	Thêm if (!mounted) return; hoặc if (dongY && mounted) — Mục 9.3
BottomSheet che hết màn hình	Thiếu mainAxisSize: MainAxisSize.min	Thêm vào Column bên trong builder — Mục 7.1
Hộp thoại nhập liệu đóng dù còn lỗi	Gọi Navigator.pop trước khi validate	Đảo thứ tự: validate xong mới pop — Mục 5.1
SnackBar chồng chất nhiều cái	Gọi showSnackBar liên tục	Gọi hideCurrentSnackBar() trước — Mục 6

Hiện tượng	Nguyên nhân	Cách xử lý
Red screen sau khi đóng hộp thoại nhập liệu	Dispose TextEditingController ngay sau await showDialog; Form vẫn còn đăng ký phụ thuộc	Tách hộp thoại thành StatefulWidget, dispose trong State.dispose() — Mục 5.3
Vuốt từ mép trái mở Drawer bị hệ thống hiểu là Back	Emulator đang ở điều hướng cử chỉ (gestural)	Chuyển sang thanh 3 nút bằng cmd overlay enable …navbar.threebutton
adb shell input text không gõ được tiếng Việt	Lệnh chỉ nhận ASCII, dấu cách phải viết %s	Dùng tên "Demo video F6" cho lần chạy tự động; dữ liệu mẫu vẫn tiếng Việt đầy đủ
Ảnh chụp trúng màn hình splash	App khởi động chậm hơn thời gian chờ cố định	Chờ tới khi ảnh screencap > 90 KB rồi thêm 2,5 giây mới chụp

Sự cố đáng nhớ nhất là dispose controller. Ban đầu app chạy đúng vài lần rồi red screen với assertion "_dependents.isEmpty". Đọc stack trace thấy lỗi nằm trong InheritedElement.debugDeactivated, không phải ở code hộp thoại — phải lần theo vòng đời widget mới hiểu. Bài học: khi gặp assertion trong framework, đừng sửa triệu chứng bằng delay; hãy tìm widget nào đang bị hủy sai thời điểm.

15. Câu hỏi ôn tập

1. Scaffold nhận những thành phần nào? Kể ít nhất bốn.

appBar, drawer, body, bottomNavigationBar, floatingActionButton (ngoài ra còn endDrawer, bottomSheet, persistentFooterButtons, backgroundColor…). Bài dùng cả năm thành phần đầu.

2. Vì sao Drawer và NavigationBar phải dùng chung một biến trạng thái?

Vì cả hai đều là cách chọn cùng một tab. Hai biến riêng sẽ lệch nhau ngay khi người dùng chọn ở một phía: Drawer đổi nhưng thanh dưới vẫn sáng mục cũ, và không có cách nào biết biến nào đúng. Một biến _tab là nguồn sự thật duy nhất, mọi nơi chỉ đọc/ghi nó.

3. showDialog trả về gì? Vì sao phải xử lý null?

Trả về Future<T?> — future hoàn tất với giá trị mà Navigator.pop truyền vào. Nó là null khi người dùng đóng hộp thoại mà không chọn nút nào (chạm ra ngoài, bấm Back). Vì vậy phải dùng ?? false hoặc kiểm tra == true trước khi dùng như bool.

4. Khi nào dùng SnackBar, khi nào dùng AlertDialog?

SnackBar khi việc đã xong và người dùng có thể muốn sửa ngay (đã xóa — HOÀN TÁC). AlertDialog khi phải hỏi trước một việc nguy hiểm hoặc cần quyết định rõ ràng (xóa, xóa tất cả). Bài dùng cả hai: hỏi trước khi xóa, báo sau khi xóa kèm Hoàn tác.

5. Vì sao phải kiểm tra mounted sau await khi còn dùng context?

Trong khoảng thời gian chờ, widget có thể bị gỡ khỏi cây. Dùng context của một State đã chết sẽ ném lỗi hoặc thao tác lên đối tượng không còn tồn tại. mounted cho biết State còn gắn với cây widget hay không; kiểm tra trước khi gọi setState hoặc ScaffoldMessenger.of(context) là bắt buộc về mặt logic, kể cả khi analyzer không phát hiện (Mục 9.3).

16. Kết luận và sản phẩm nộp

Bài lab hoàn thành đủ yêu cầu: Scaffold đủ năm thành phần; Drawer và NavigationBar dùng chung _tab; showDialog<bool> xử lý đúng null; hộp thoại nhập liệu validate và không đóng khi

sai; SnackBar Hoàn tác chèn lại đúng vị trí; PopupMenuButton lọc danh sách đúng và tab Thống kê vẫn đếm trên dữ liệu gốc. Ba bài nâng cao NC1–NC3 đều chạy trên máy ảo và có ảnh minh chứng. flutter analyze sạch và 7 widget test đều đạt.

- Repo GitHub (public): https://github.com/nvquangggit-sys/F6_231A290134
- Mã nguồn: lib/main.dart (khoảng 470 dòng); pubspec.yaml; README.md mô tả cách chạy và danh sách ảnh.
- Kiểm thử: test/widget_test.dart — 7 widget test, flutter test: All tests passed!
- Minh chứng: docs/screenshots (22 ảnh máy ảo, 1 ảnh cửa sổ analyze và bản cắt) và docs/logs (flutter_analyze.txt, flutter_test.txt, phan_tich.txt).
- Video demo: docs/F6_231A290134_demo.mp4 — 1 phút 08 giây, đủ trình tự Drawer → đổi tab → thêm việc → xóa có hỏi lại → Hoàn tác → lọc → NC2 → NC3.
- Báo cáo: tệp này (DOCX) và bản PDF cùng tên.

Bài học lớn nhất không nằm ở widget nào: đó là sự khác nhau giữa “dữ liệu gốc” và “cách nhìn dữ liệu”. Cùng một danh sách, tab Thống kê phải đếm trên _ds còn màn hình phải vẽ từ _hienThi; nhầm hai thứ này thì mọi con số đều sai dù giao diện vẫn đẹp. Sang các bài sau, khi có thêm cơ sở dữ liệu và trạng thái phức tạp hơn, ranh giới đó còn quan trọng hơn nữa.